

Buffer overflow mitigations

- Avoid unsafe functions
- Stack canaries
- Separate control stack
- Memory writable or executable, not both (W^X)
- Address space layout randomization

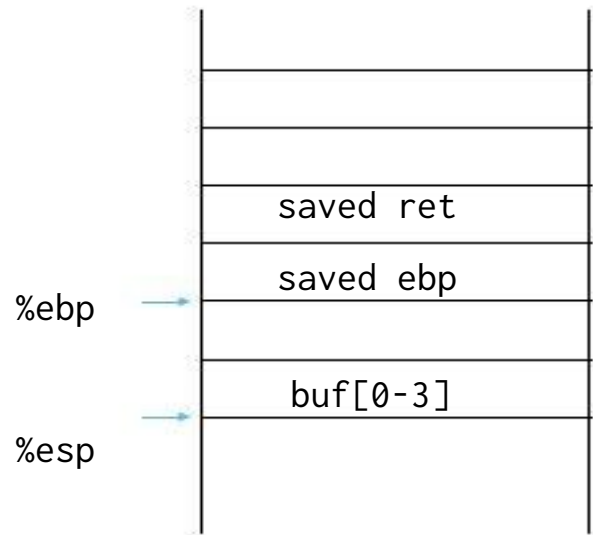
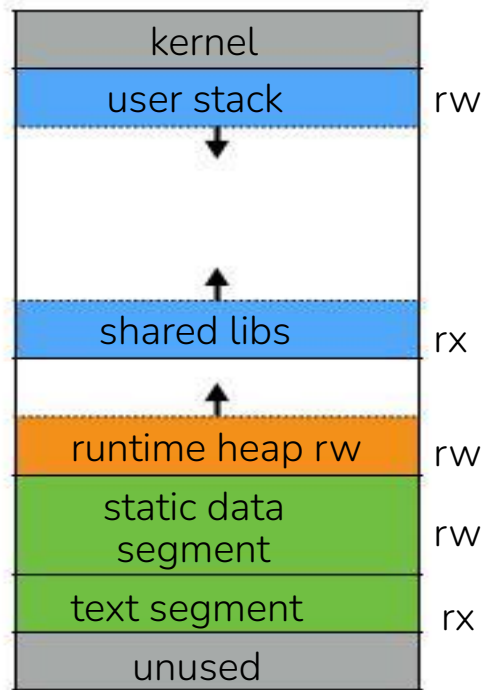
W^X: write XOR execute

- **Goal:** Prevent execution of shell code from the stack

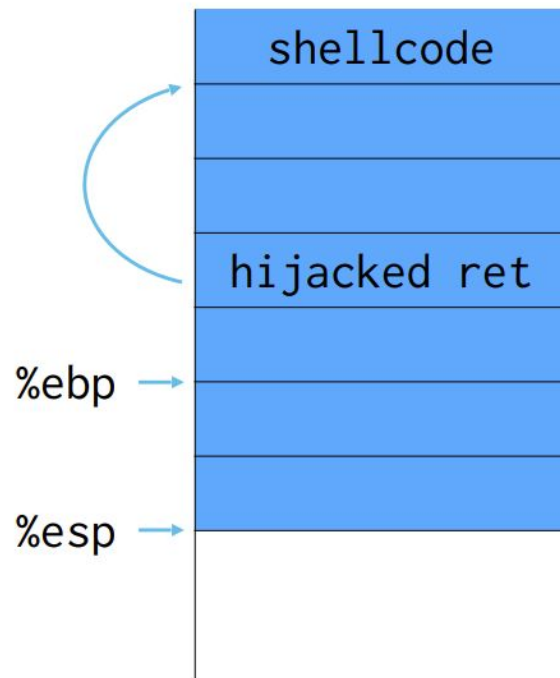
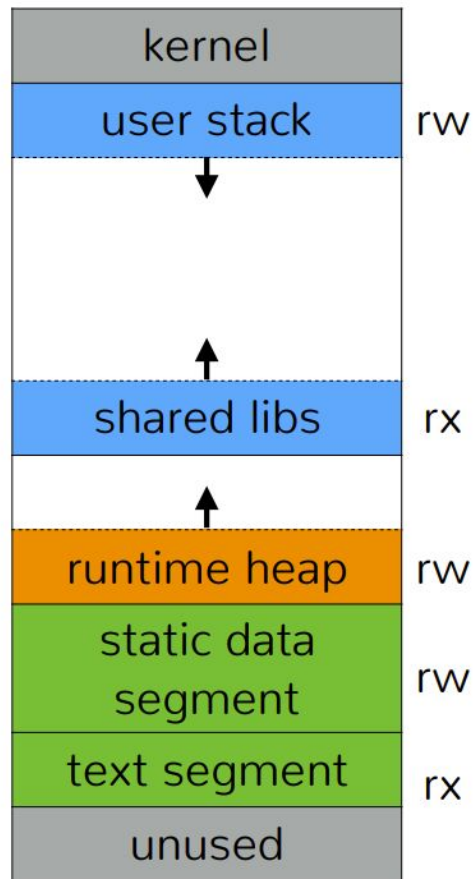
Insight: Use memory page permission bits

- - Use MMU to ensure memory cannot be both writeable and executable at the same time
- Many names for same idea:
 - XN: eXecute Never
 - W^X: Write XOR eXecute
 - DEP: Data Execution Prevention

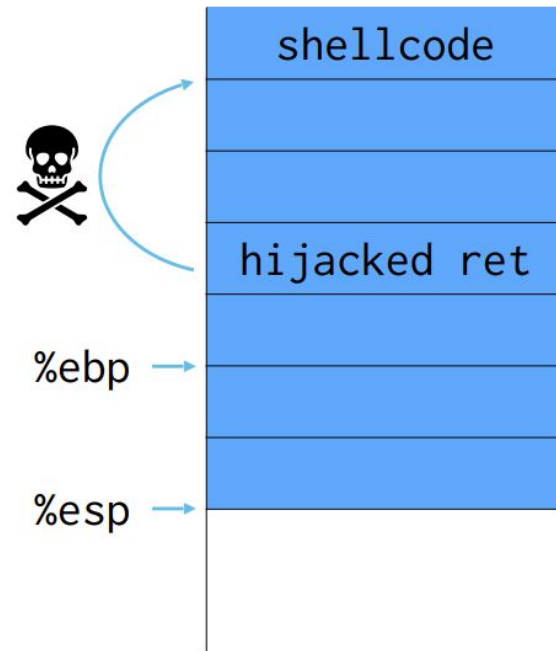
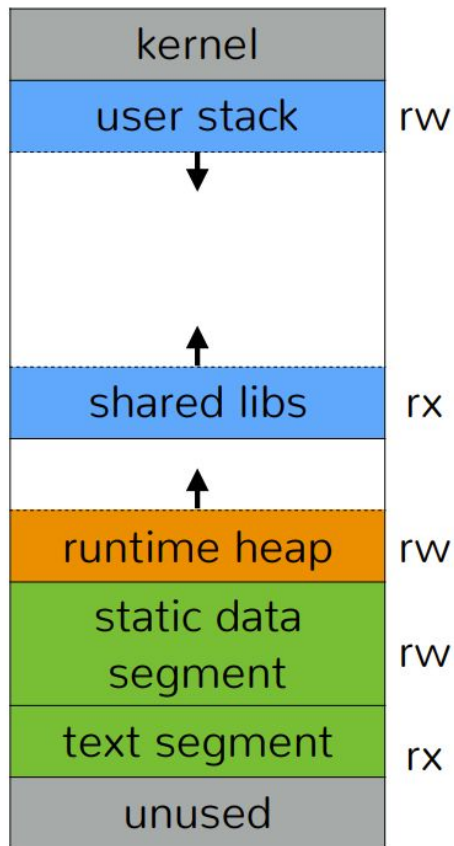
Recall our memory layout



Recall our memory layout



Recall our memory layout



W^X tradeoffs

- **Easy to deploy:** No code changes or recompilation
- **Fast:** Enforced in hardware
 - Downside: What do you do on embedded devices?
- Some pages need to be both writeable and executable
 - Why?

How can we defeat W^X?

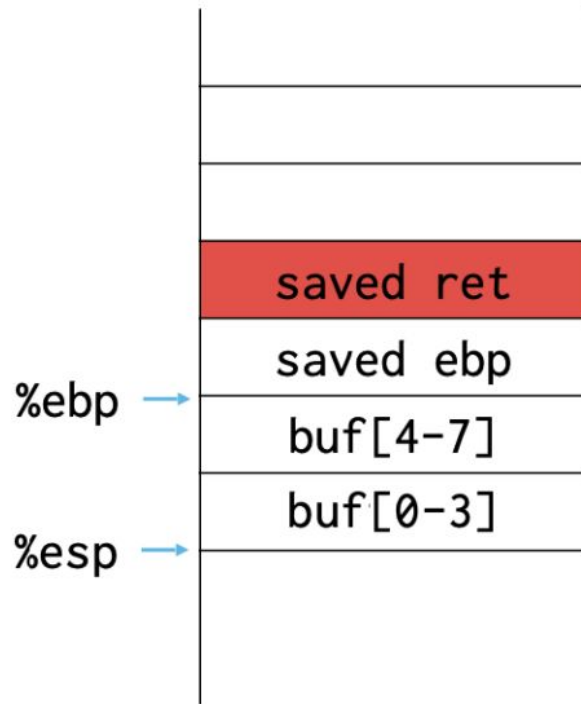
- Can still write to return address stored on the stack
 - Jump to existing code

Search executable for code that does what you want

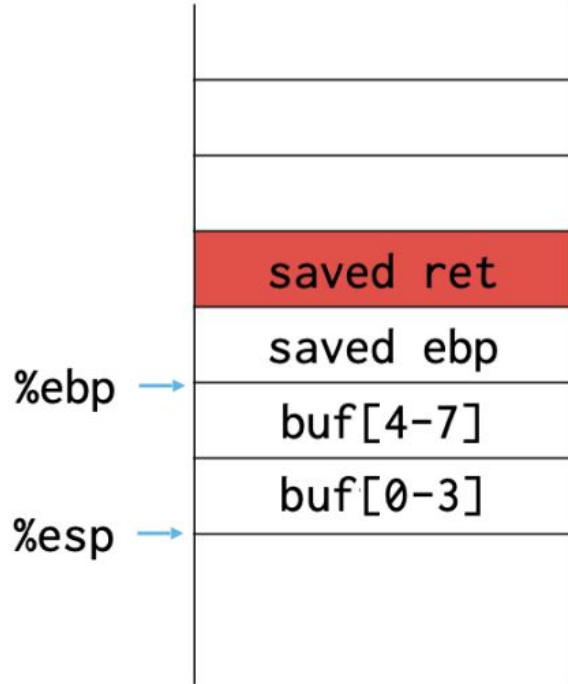
- - E.g. if program calls `system("/bin/sh")` you're done
 - libc is a good source of code (return-into-libc attacks)

Redirecting control flow to system()

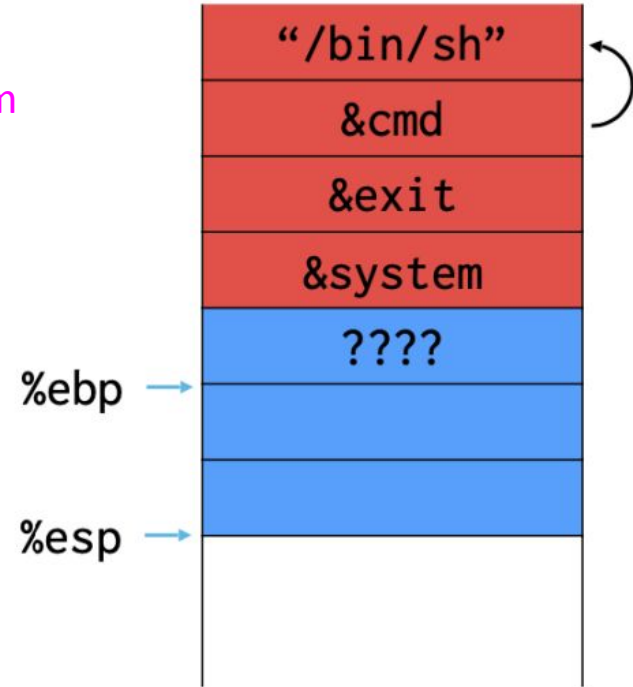
- We redirected control flow earlier to `foo()`
- Calling `system()` is the same, but need to have argument string `"/bin/sh"` on stack.



Redirecting control flow to system()



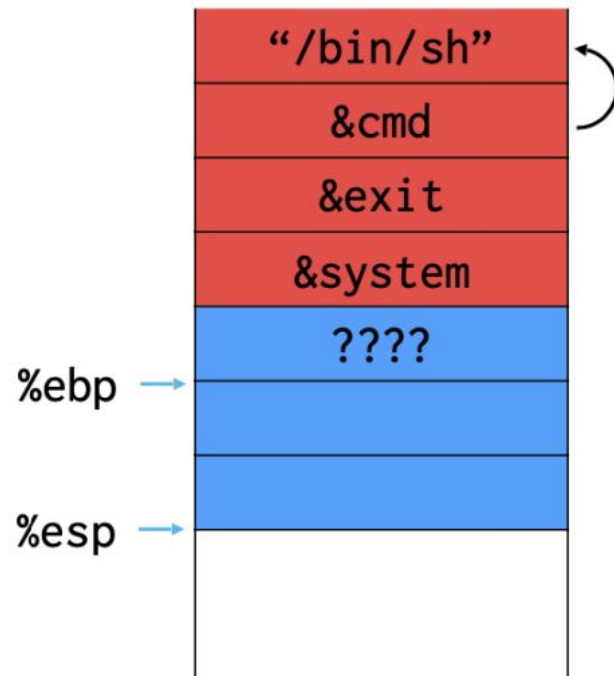
Remember: &system
is the address of
system()



Redirecting control flow to system()

leave → movl %ebp, %esp
 pop %ebp

ret → popl %eip



Redirecting control flow to system()

After leave

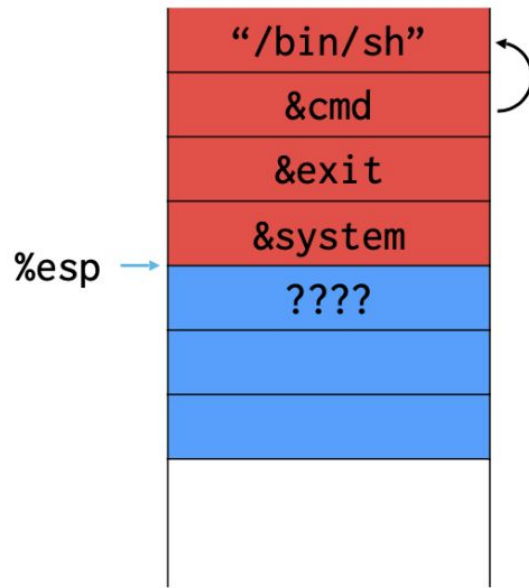
leave

→ `movl %ebp, %esp`
`pop %ebp`

ret

→ `popl %eip`

`%ebp` → ????



Redirecting control flow to system()

After ret

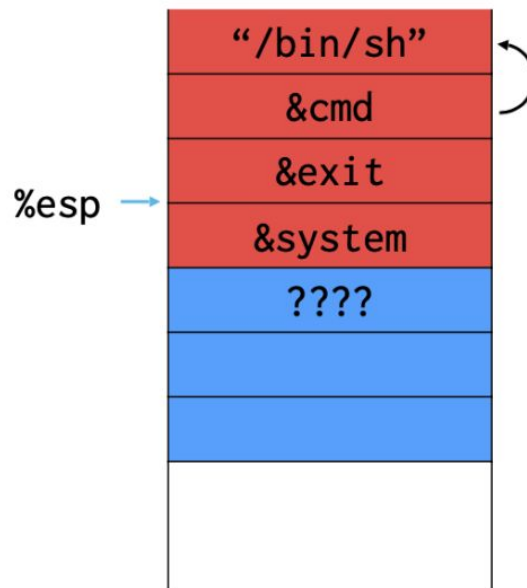
leave

→ `movl %ebp, %esp`
`pop %ebp`

ret

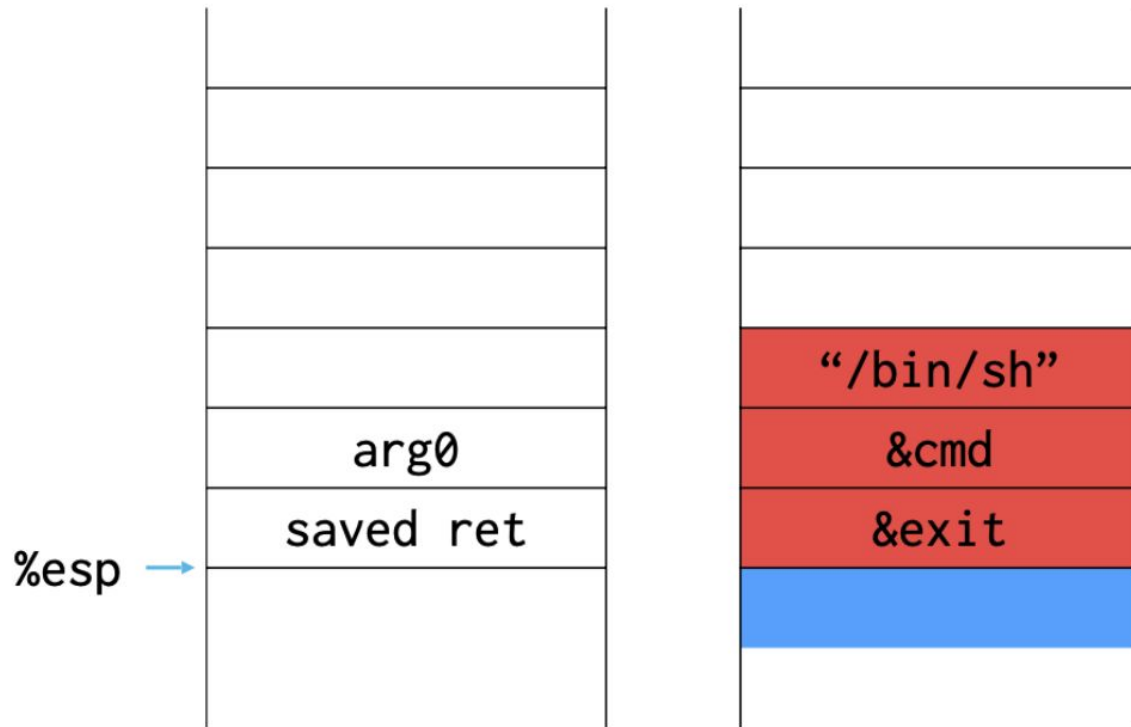
→ `popl %eip`

`%ebp` → ????



`%eip` → `&system`

To system this looks like a normal call



But I want to execute shellcode, not just call `system()`!

Can we inject code?

- Just-in-time compilers produce data that becomes executable code

JIT spraying:

- 1. Spray heap with shellcode (and NOP slides)
 2. Overflow code pointer to spray area

What does JIT shellcode look like?

```
1  var g1 = 0;
2  ...
3  var g7 = 0;
4
5  for (var i=0; i<100000; ++i) {
6      g1 = 50011;    // pop ebx; ret;
7      g2 = 50009;    // pop ecx; ret;
8      g3 = 12828721; // xor eax, eax; ret;
9      g4 = 12811696; // mov 0x7d, al; ret;
10     g5 = 12833329; // xor edx, edx; ret;
11     g6 = 12781490; // mov 0x7, dl; ret;
12     g7 = 12812493; // int 0x80; ret;
13 }
```

The Devil is in the Constants: Bypassing Defenses in Browser JIT Engines

Michalis Athanasakis	Elias Athanasopoulos	Michalis Polychronakis	Georgios Portokalidis	Sotiris Ioannidis
FORTH, Greece	FORTH, Greece	Stony Brook University	Stevens Institute of Tech.	FORTH, Greece
michath@ics.forth.gr	clathan@ics.forth.gr	mikepo@cs.stonybrook.edu	gportoka@stevens.edu	sotiris@ics.forth.gr

Buffer overflow mitigations

- Avoid unsafe functions
- Stack canaries
- Separate control stack
- Memory writable or executable, not both (W^X)
- $\rightarrow$ Address space layout randomization (ASLR)

Address Space Layout Randomization (ASLR)

- Traditional exploits need precise addresses
 - stack-based overflows: shellcode
 - return-into-libc: library addresses

Insight: Make it harder for attacker to

- guess location of shellcode/libc by randomizing the address of different memory regions

How much do we randomize?

32-bit PaX ASLR (x86)

Stack:



Mapped area:



Executable code, static variables, and heap:



ASLR Tradeoffs

- **Intrusive:** Need compiler, linker, loader support
 - Process layout must be randomized
 - Programs must be compiled to not have absolute jumps
- **Incurs overhead:** Increases code size and performance overhead
- Also mitigates heap-based overflow attacks

When do we randomize?

Many options.

- At boot?
- At compile/link time?
- At run/load time?
- On fork?

What's the tradeoff?

How can we defeat ASLR?

- `-fno-pie` binaries have fixed code and data addresses
 - Enough to carry out control flow hijacking attacks
- Each region has random offset, but layout is fixed
 - Single address in a region leaks every address in region
- Brute force for 32-bit binaries and/or pre-fork binaries
- Heap spray for 64-bit binaries

Buffer overflow mitigations

- Avoid unsafe functions
- Stack canaries
- Separate control stack
- Memory writable or executable, not both (W^X)
- Address space layout randomization (ASLR)

None are perfect, but in practice they raise the bar dramatically.